Simulador de Sistema Hospitalario
Proyecto: Etapa 2

Equipo 2
Programación Orientada a Objetos, Grupo 7
Universidad Nacional Autónoma de México

26 de octubre de 2026

Índice

1. Introducción **3**
1.1. Objetivos 3

2. Marco teórico **3**

3. Desarrollo **4**
3.1. Paquetes y estructuras 4
3.2. Registro y atención 4
3.3. Turnos y supervisión 4
3.4. Algoritmos y diagramas 5

4. Compilación y extra **5**

5. Resultados **6**
5.1. Demostración interactiva 6
5.2. Validación 6

6. Conclusiones **7**

A. Diagramas y pseudocódigos por clase **8**
A.1. Main 9
A.2. Sistema 12
A.3. AtencionMedica 15
A.4. Paciente 18
A.5. PacienteGeneral 21
A.6. PacienteEspecialidad 24
A.7. Medico 27
A.8. MedicoGeneral 30
A.9. MedicoEspecialista 33
A.10.Enfermero 36
A.11.EnfermeroEspecialista 39
A.12.JefeEnfermeria 42

1. Introducción

El sistema representa el registro y la atención de pacientes en hospitales independientes. Conserva las reglas de la primera etapa: hasta diez pacientes por médico, una consulta activa por médico y hasta tres pacientes de la misma especialidad por enfermero. La segunda etapa organiza estas responsabilidades mediante paquetes, encapsulamiento, herencia, clases abstractas, una interfaz y polimorfismo.

El programa se utiliza desde un menú. Primero se crea el hospital, después sus especialidades y por último el personal y los pacientes. Siempre se muestran el hospital y el paciente activos. También se pueden precargar datos de demostración y cambiar de hospital sin mezclar sus registros. En cualquier captura, cero cancela la operación sin guardar datos incompletos; en el menú principal, cero sale. Al finalizar una opción se presiona Enter y se limpia la pantalla.

1.1. Objetivos

Registrar personas y especialidades por identificador, asignar atención compatible, realizar consultas y tratamientos, consultar listas descendentes y coordinar enfermería por área. La demostración debe permitir explicar las tres jerarquías y las reglas que protegen su estado.

2. Marco teórico

El encapsulamiento mantiene los atributos privados y concentra las validaciones en constructores y métodos. Las listas públicas se entregan como copias no modificables y el catálogo como un mapa no modificable. Las cédulas y especialidades asignadas al personal son finales: no se modifican después de construir el objeto, porque cambiarían su identidad profesional o la compatibilidad de sus pacientes. Los nombres, el motivo de consulta, el turno y el tratamiento sí pueden actualizarse mediante métodos. Los contadores no tienen setters y solo cambian con registros aceptados.

Las clases abstractas `Paciente`, `Medico` y `Enfermero` contienen la información compartida. Sus subclases utilizan `super` y sobrescriben las operaciones que dependen del tipo real. Cada subclase incorpora un dato propio: motivo de consulta, servicio general, especialidad o área de supervisión. La interfaz `AtencionMedica` declara `darTratamiento(Paciente)` y es implementada por médicos y enfermeros.

El polimorfismo estático aparece en las sobrecargas de `Sistema.asignarPaciente`. La llamada con `PacienteGeneral` o `PacienteEspecialidad` busca un médico; la llamada con `MedicoGeneral` o `MedicoEspecialista` busca un paciente. La referencia base `Paciente` selecciona la sobrecarga que busca enfermería. El polimorfismo dinámico aparece en `buscarMedico`, `darConsulta`, `getEspecialidad` y `darTratamiento`: una referencia base ejecuta el comportamiento del objeto real. Los jefes rechazan el tratamiento directo porque su función es coordinar.

3. Desarrollo

3.1. Paquetes y estructuras

La fuente principal está en `src/hospital`. Se separan los paquetes `hospital.sistema`, `hospital.personal`, `hospital.pacientes` y `hospital.interfaces`. `Main` pertenece a `hospital`. Los imports conectan las clases y el menú emplea el nombre completo `hospital.interfaces.AtencionMedica` al proporcionar tratamiento.

Cada hospital conserva una sola lista por jerarquía: `List<Medico>`, `List<Enfermero>` y `List<Paciente>`. Las listas pueden contener las distintas subclases. El catálogo es `Map<Integer, String>`, implementado con `LinkedHashMap` para mantener el orden de registro. No se utiliza una lista de listas. El ID 1 identifica medicina general; los IDs 2 a 99 se eligen al crear especialidades y pueden no ser consecutivos. Cada selección muestra el ID y su nombre y rechaza números que no existan.

3.2. Registro y atención

Los registros verifican nombres, cédulas, hospital de pertenencia y especialidades del catálogo. Las cédulas no se repiten dentro del hospital, incluso entre médicos y enfermeros. Un objeto no se registra en dos hospitales distintos.

El paciente inicia en `EN_ESPERA`. Al solicitar consulta se utiliza su médico asignado o se busca uno compatible con cupo y sin consulta activa. El médico valida que el paciente esté en su lista y libre para consulta; entonces cambia a `EN_CONSULTA`. Un segundo paciente no puede ocupar la misma consulta.

El tratamiento busca un enfermero de la misma especialidad con menos de tres pacientes. Si el paciente necesita medicamento controlado, el enfermero solicita autorización al supervisor de su área. Solamente cuando se cumplen estas condiciones se asigna enfermero, se guarda tratamiento, se cambia a `EN_TRATAMIENTO` y se libera la consulta médica. Si falta cupo o supervisor, se conserva la consulta y no se pierde la asignación. La atención general utiliza un enfermero especialista del área 1.

Los métodos `verListaPacientes` ordenan una copia por nombre completo de Z a A y muestran especialidad. El menú solicita escribir apellidos antes de los nombres para que el orden corresponda a lo requerido.

3.3. Turnos y supervisión

Cada enfermero tiene un turno booleano: `false` significa matutino y `true`, vespertino. Cada jefe pertenece a una especialidad y solo supervisa enfermeros especialistas de esa área y hospital. El equipo no incluye al propio jefe ni a otros supervisores. Se permiten como máximo dos jefes por especialidad y hospital.

El jefe puede ver los turnos, cambiar el turno de su equipo y trasladar un paciente entre dos enfermeros de su área. Antes del traslado se verifica que el paciente esté en tratamiento, que pertenezca al enfermero actual y que el destino sea distinto, compatible y tenga cupo. Si alguna condición falla, no cambia ninguna lista. Si todas se cumplen, el paciente sale de la lista anterior, entra a la nueva y actualiza su enfermero.

Cada paciente recibe al construirse el booleano `necesitaMedicamento` mediante `Math.random() <0.5`. La necesidad se muestra en pantalla y no cambia al consultar el menú. El enfermero comunica este requerimiento al supervisor; en esta simulación toda solicitud válida del área se autoriza. El programa conserva por separado el requerimiento y la autorización. Si no se necesita medicamento, no se solicita autorización. Esta regla representa el comportamiento académico solicitado.

3.4. Algoritmos y diagramas

El flujo central es: seleccionar hospital, registrar o elegir paciente, buscar atención compatible, validar consulta, solicitar enfermería y autorizar medicamento si corresponde. La Figura 1 reúne las tres jerarquías, sus clases abstractas y la interfaz. Los diagramas individuales y flujos se incluyen en el anexo; cada imagen tiene su archivo editable DrawIO.

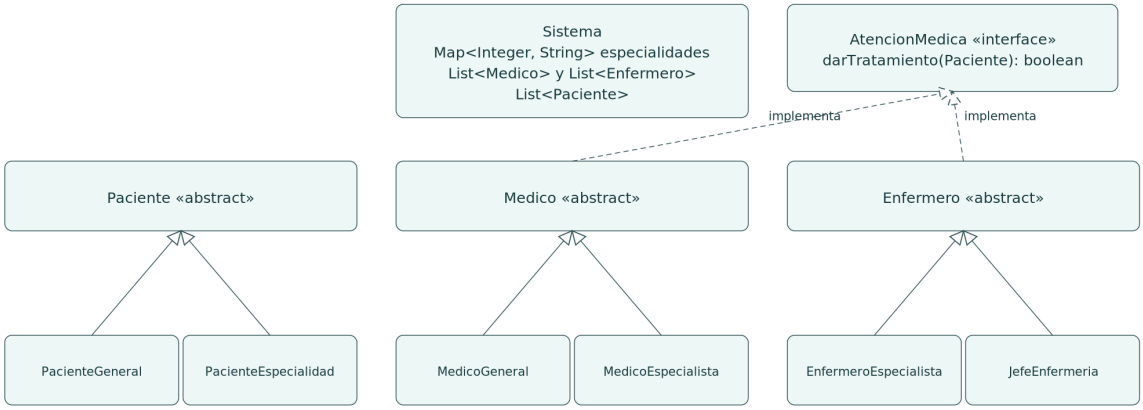


Figura 1: Jerarquías y estructuras del sistema hospitalario.

Los doce pseudocódigos se encuentran en `doc/Pseudocodigos`; se incorporan al anexo para documentar los algoritmos junto a sus diagramas.

4. Compilación y extra

Se utiliza Java 17 o posterior, sin dependencias externas al JDK. Desde `Proyecto2/`:

```
./compilar.sh
java -jar lib/hospital-fase2.jar
```

El script compila los paquetes, construye `lib/atencion-medica.jar`, recompila la aplicación usando ese JAR en el classpath y produce el ejecutable `lib/hospital-fase2.jar`. El ejecutable incluye también la interfaz para funcionar por sí solo. Los archivos compilados

quedan en `test/hospital`. El JAR separado de la interfaz cubre el extra indicado en la segunda etapa.

La documentación conjunta se abre en `doc/Javadoc/General/index.html`; se conservan también las páginas individuales de cada clase. Para repetir las pruebas:

```
java -cp test PruebasHospital
```

5. Resultados

5.1. Demostración interactiva

La opción D crea un hospital nuevo con medicina general (ID 1), cardiología (ID 2) y neurología (ID 7). Registra tres médicos, seis enfermeros especialistas, tres supervisores y seis pacientes en espera. Cada área tiene un enfermero matutino, uno vespertino y un supervisor. La paciente activa inicial es María López García, registrada como `López García, María`.

La secuencia de demostración consiste en solicitar consulta con 6, indicar tratamiento con 7, consultar el detalle con 14, cambiar turno con 16 y trasladar al paciente con 17. La necesidad de medicamento puede variar entre ejecuciones, pero el supervisor del área autoriza automáticamente cuando hace falta. El resultado es un paciente en tratamiento con la indicación capturada, un turno actualizado y una asignación de enfermería coherente. El

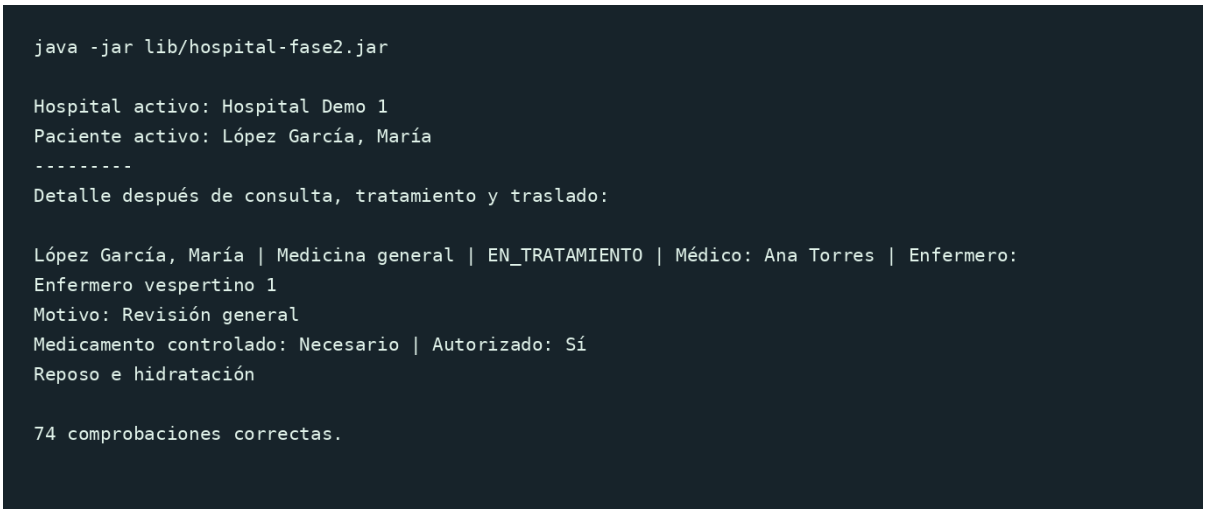


Figura 2: Salida real del menú y detalle del paciente durante la demostración.

transcript completo de la ejecución está en `doc/Capturas/ejecucion-main.txt`.

5.2. Validación

La prueba `test/PruebasHospital.java` realiza 74 comprobaciones reproducibles. Fija la necesidad de medicamento antes de las consultas para comprobar ambos caminos sin depender del azar. Verifica registros duplicados, especialidades inexistentes, separación de

hospitales, cupos de 10 y 3, una consulta activa, dos supervisores como máximo, cambios de turno, traslado válido e inválido y rechazo de modificaciones externas a las colecciones.

Se comprueba que un paciente que requiere medicamento siga en consulta si falta supervisor y pase a tratamiento después de registrar uno compatible. También se verifica que un paciente que no lo requiere pueda recibir tratamiento sin autorización. El resultado de las pruebas está en `doc/Capturas/pruebas.txt`.

Las pruebas del menú recorren tanto la demo como un registro manual con ID 23, cambian de hospital y paciente, modifican turnos, trasladan pacientes y cancelan capturas de personal y pacientes sin dejar registros parciales.

6. Conclusiones

Las reglas se mantienen en los objetos y el menú se limita a capturar datos e invocar operaciones. El mapa permite conservar una identidad explícita para cada especialidad, mientras las listas de clases base muestran el uso de herencia y polimorfismo. La supervisión tiene funciones concretas: administrar turnos, coordinar asignaciones y autorizar medicamentos dentro del área correspondiente.

Las pruebas confirman que los rechazos conservan los estados y asociaciones previos. La distribución incluye fuente, clases compiladas, JAR ejecutable, JAR de interfaz, Javadoc, diagramas, pseudocódigos y presentación de la misma implementación.

Referencias

[1] Programación Orientada a Objetos, Grupo 7. *Proyecto – Etapa 1: Simulador de Sistema Hospitalario*. Especificaciones del curso, 2026. Archivo `doc/etapa-1.pdf`.

[2] Programación Orientada a Objetos, Grupo 7. *Proyecto – Etapa 2: Simulador de Sistema Hospitalario*. Especificaciones del curso, 2026. Archivo `doc/etapa-2.pdf`.

[3] Oracle. *Java Platform, Standard Edition 17: API Specification*. `https://docs.oracle.com/en/java/javase/17/docs/api/`.

A. Diagramas y pseudocódigos por clase

A.1. Main

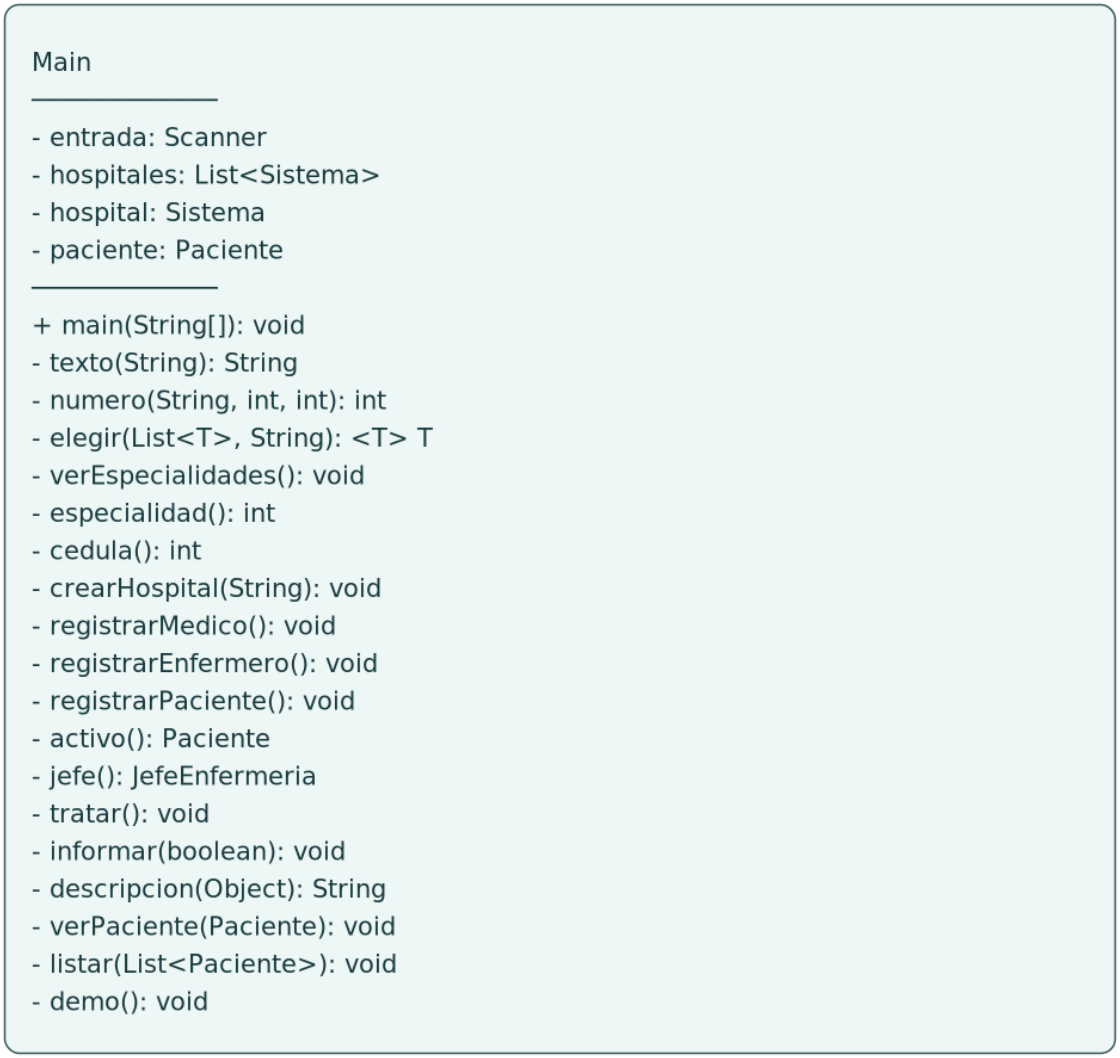


Figura 3: Diagrama de clase Main.

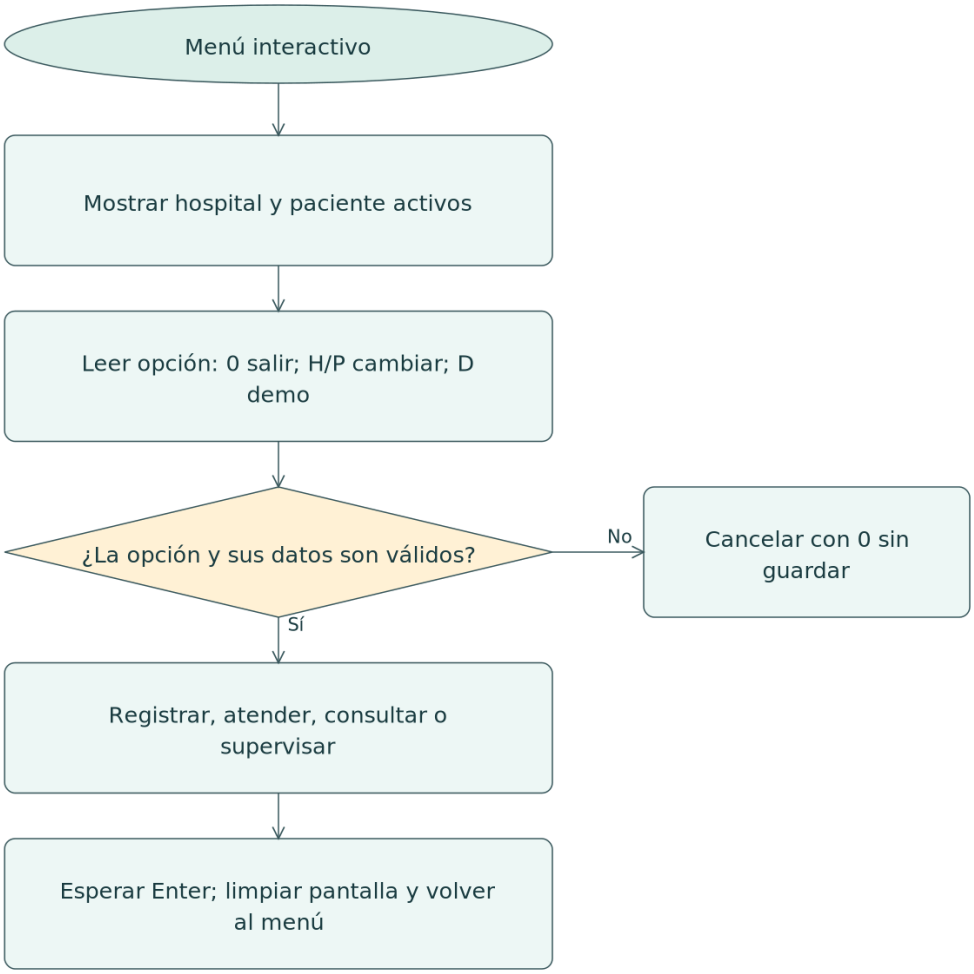


Figura 4: Flujo principal de Main.

Pseudocódigo de Main

```
ALGORITMO Main
MENÚ INTERACTIVO

  Mostrar hospital y paciente activos
  Leer opción: 0 salir; H/P cambiar; D demo
  ¿La opción y sus datos son válidos?
  Registrar, atender, consultar o supervisar
  Esperar Enter; limpiar pantalla y volver al menú
  Si no se cumplen las condiciones: Cancelar con 0 sin guardar
FIN
```

A.2. Sistema



Figura 5: Diagrama de clase Sistema.

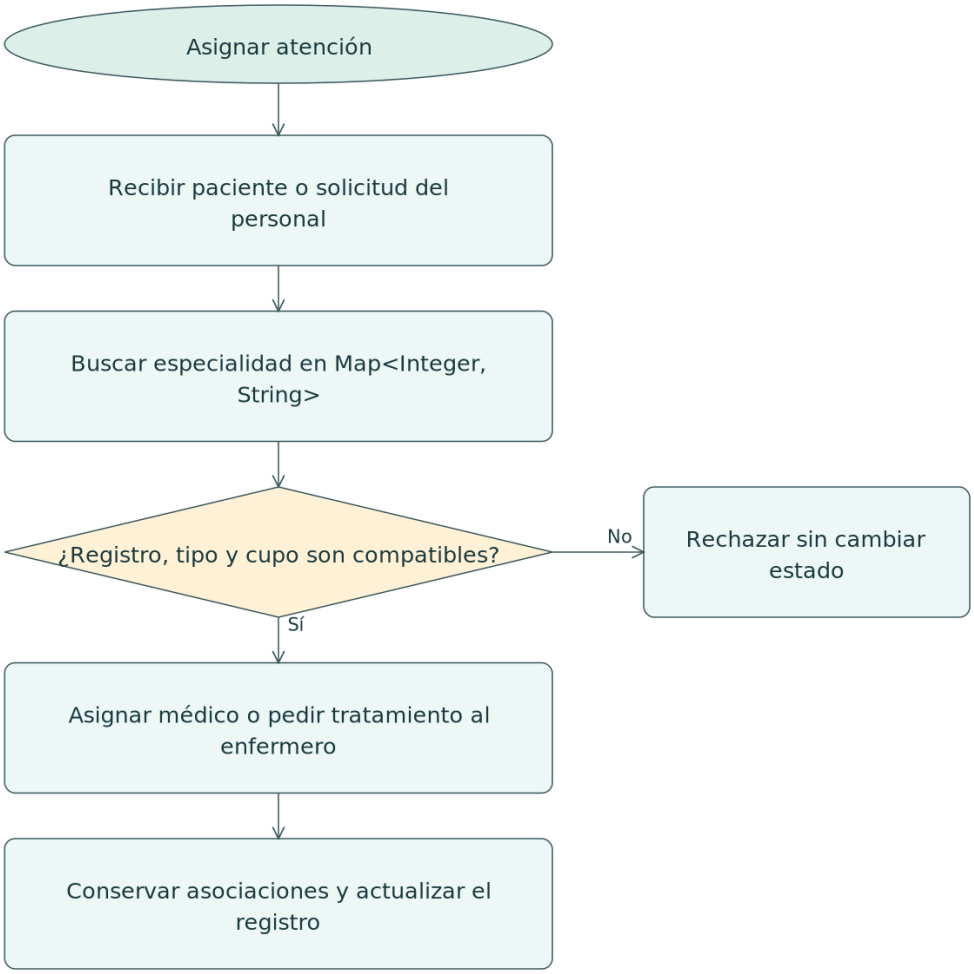


Figura 6: Flujo principal de Sistema.

Pseudocódigo de Sistema

```
ALGORITMO Sistema
ASIGNAR ATENCIÓN

  Recibir paciente o solicitud del personal
  Buscar especialidad en Map<Integer, String>
  ¿Registro, tipo y cupo son compatibles?
  Asignar médico o pedir tratamiento al enfermero
  Conservar asociaciones y actualizar el registro
  Si no se cumplen las condiciones: Rechazar sin cambiar estado
FIN

REGISTRAR ESPECIALIDAD
  Validar ID entre 2 y 99 y nombre no vacío ni repetido.
  Guardar ID y nombre en Map. El ID 1 corresponde a medicina general.

REGISTRAR JEFE
  Contar solamente los jefes de su especialidad en este hospital.
  Rechazar si ya hay dos; en otro caso registrar.
```

A.3. AtencionMedica



Figura 7: Diagrama de clase AtencionMedica.

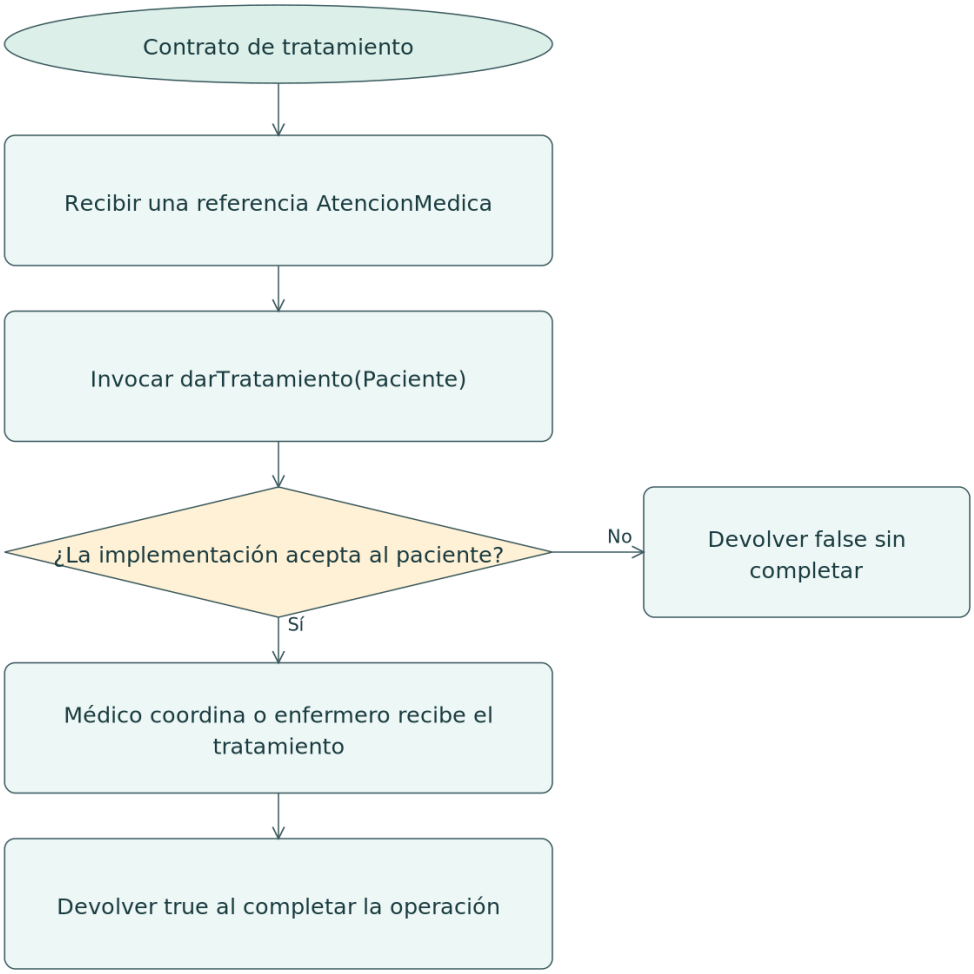


Figura 8: Flujo principal de AtencionMedica.

Pseudocódigo de AtencionMedica

```
ALGORITMO AtencionMedica
CONTRATO DE TRATAMIENTO

  Recibir una referencia AtencionMedica
  Invocar darTratamiento(Paciente)
  ¿La implementación acepta al paciente?
  Médico coordina o enfermero recibe el tratamiento
  Devolver true al completar la operación
  Si no se cumplen las condiciones: Devolver false sin completar
FIN
```

A.4. Paciente

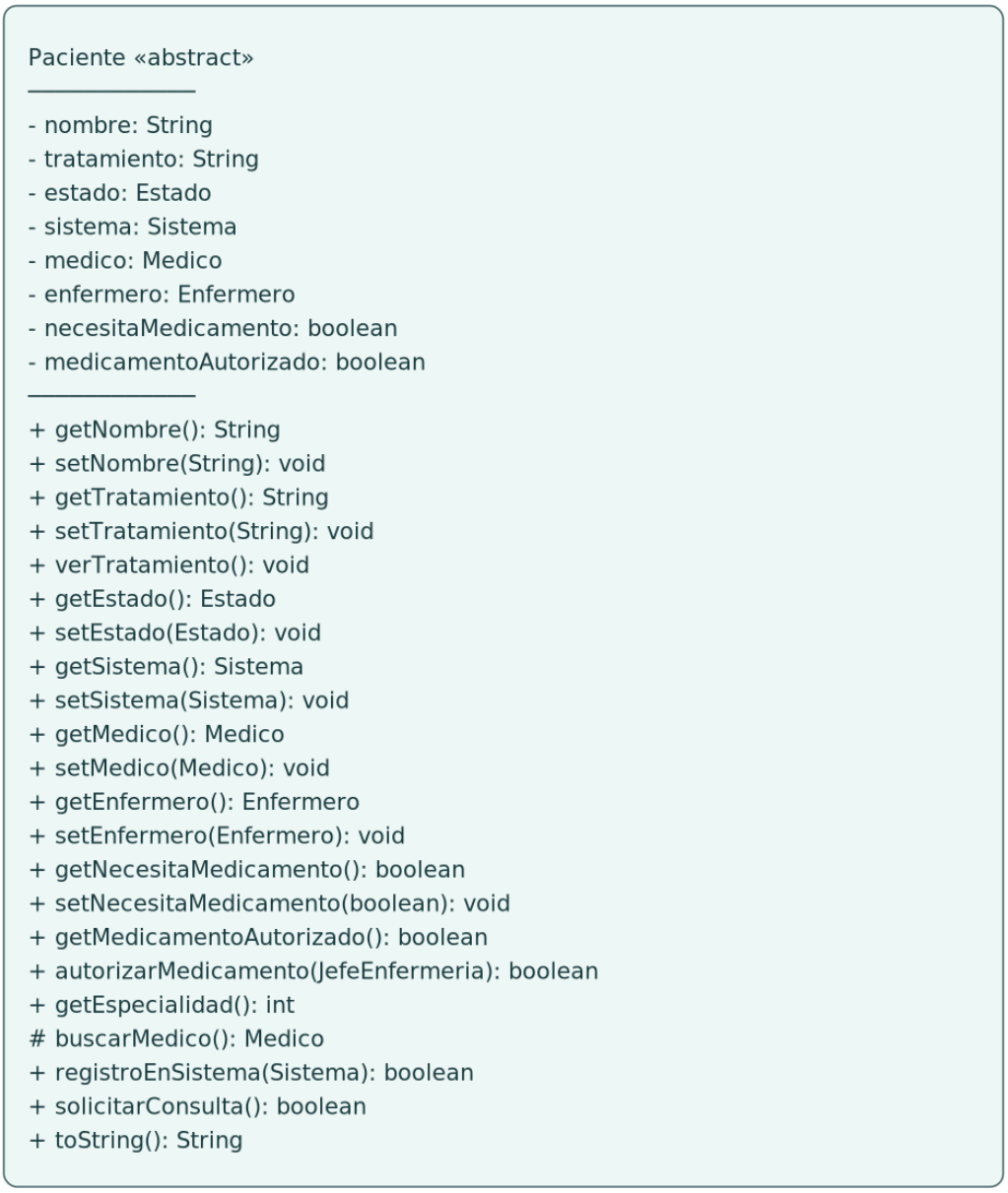


Figura 9: Diagrama de clase Paciente.

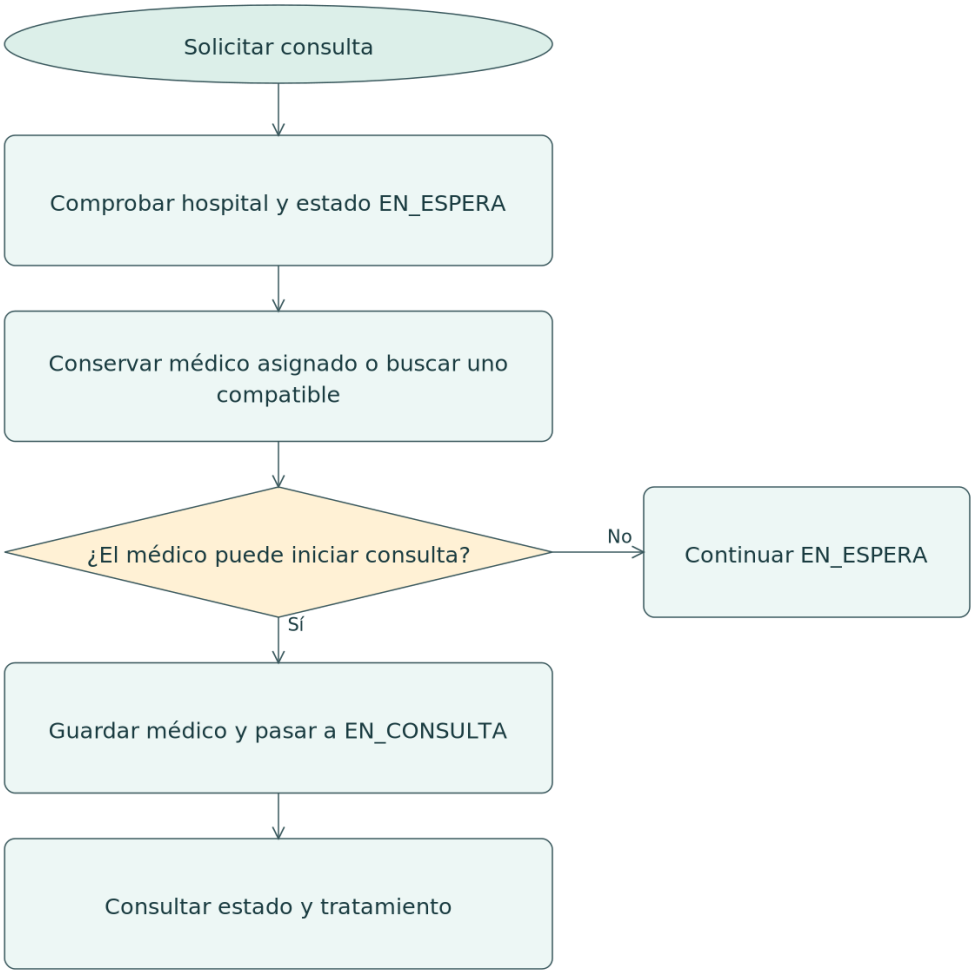


Figura 10: Flujo principal de Paciente.

Pseudocódigo de Paciente

```
ALGORITMO Paciente
SOLICITAR CONSULTA

  Comprobar hospital y estado EN_ESPERA
  Conservar médico asignado o buscar uno compatible
  ¿El médico puede iniciar consulta?
  Guardar médico y pasar a EN_CONSULTA
  Consultar estado y tratamiento
  Si no se cumplen las condiciones: Continuar EN_ESPERA
FIN
```

A.5. PacienteGeneral

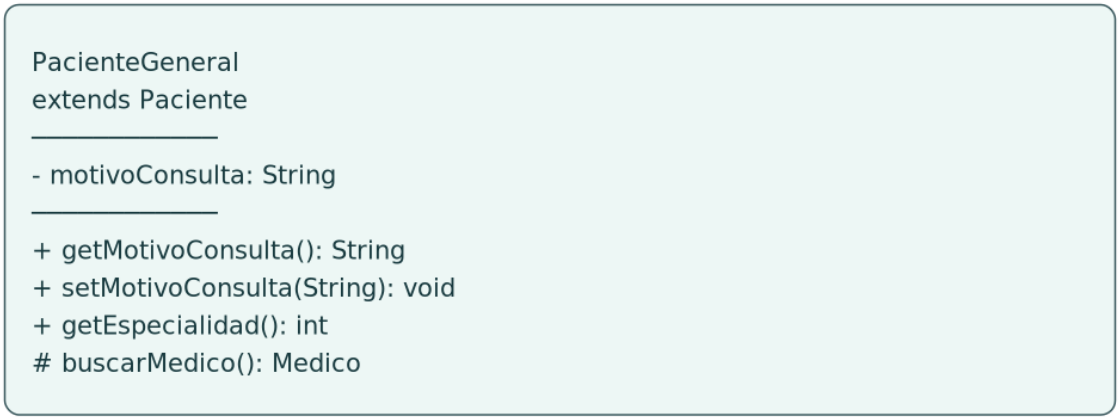


Figura 11: Diagrama de clase PacienteGeneral.

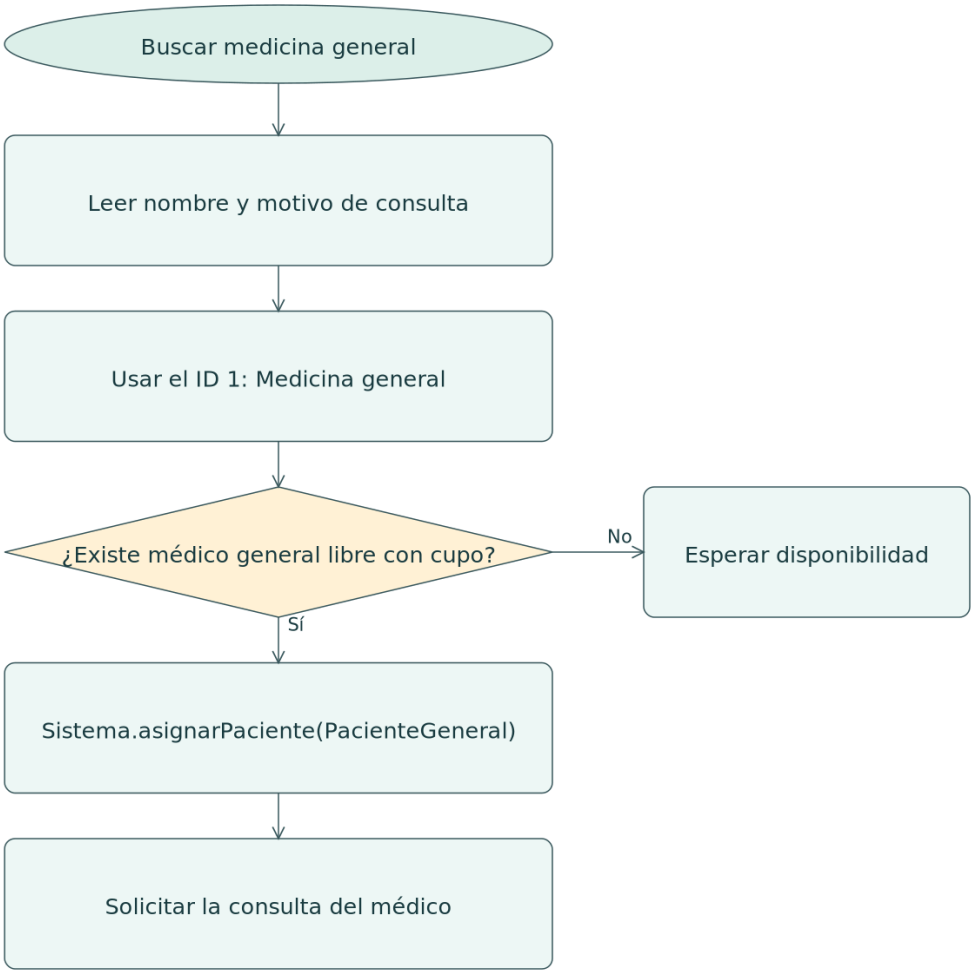


Figura 12: Flujo principal de PacienteGeneral.

Pseudocódigo de PacienteGeneral

```
ALGORITMO PacienteGeneral
BUSCAR MEDICINA GENERAL

  Leer nombre y motivo de consulta
  Usar el ID 1: Medicina general
  ¿Existe médico general libre con cupo?
  Sistema.asignarPaciente(PacienteGeneral)
  Solicitar la consulta del médico
  Si no se cumplen las condiciones: Esperar disponibilidad
FIN
```

A.6. PacienteEspecialidad

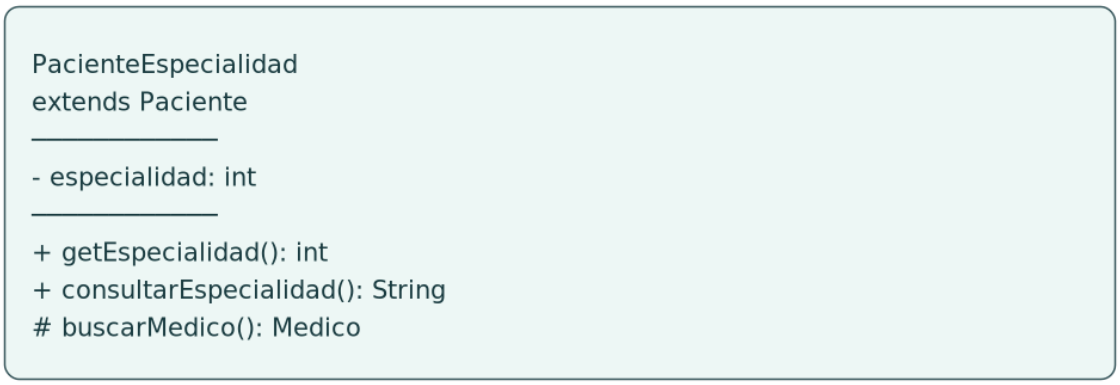


Figura 13: Diagrama de clase PacienteEspecialidad.

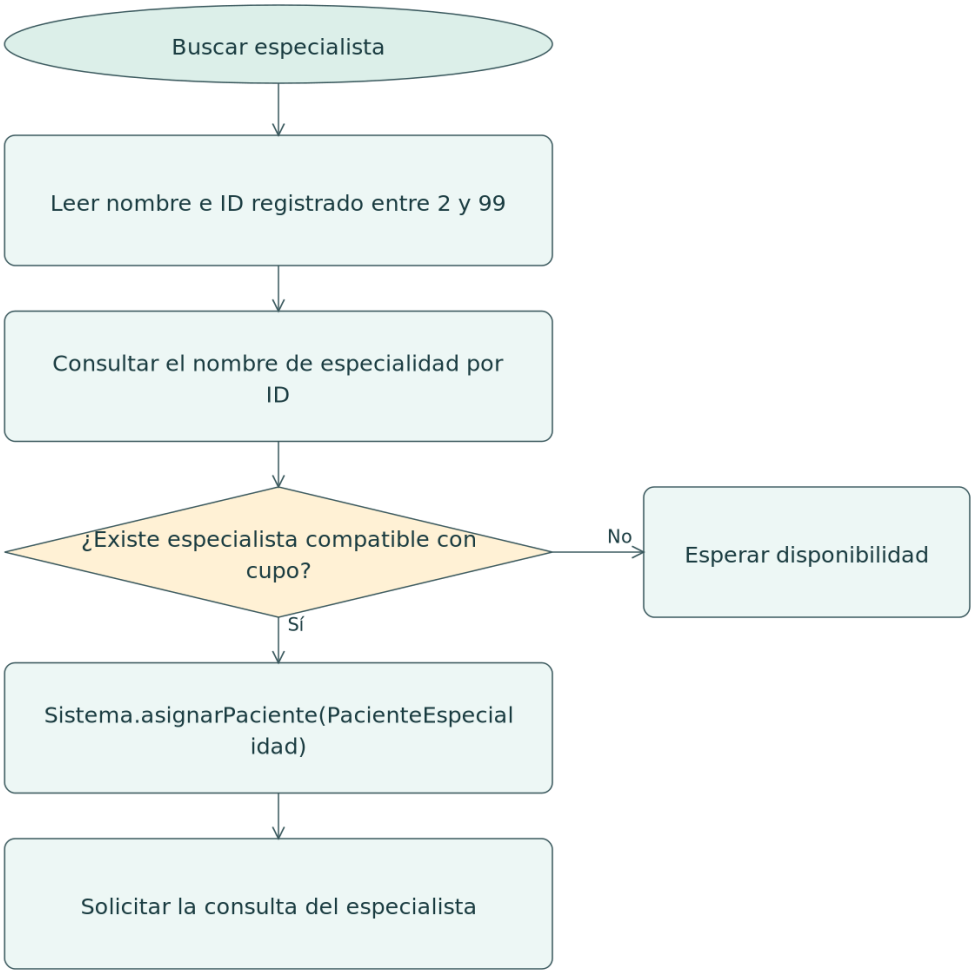


Figura 14: Flujo principal de PacienteEspecialidad.

Pseudocódigo de PacienteEspecialidad

```
ALGORITMO PacienteEspecialidad
BUSCAR ESPECIALISTA

  Leer nombre e ID registrado entre 2 y 99
  Consultar el nombre de especialidad por ID
  ¿Existe especialista compatible con cupo?
  Sistema.asignarPaciente(PacienteEspecialidad)
  Solicitar la consulta del especialista
  Si no se cumplen las condiciones: Esperar disponibilidad
FIN
```

A.7. Medico

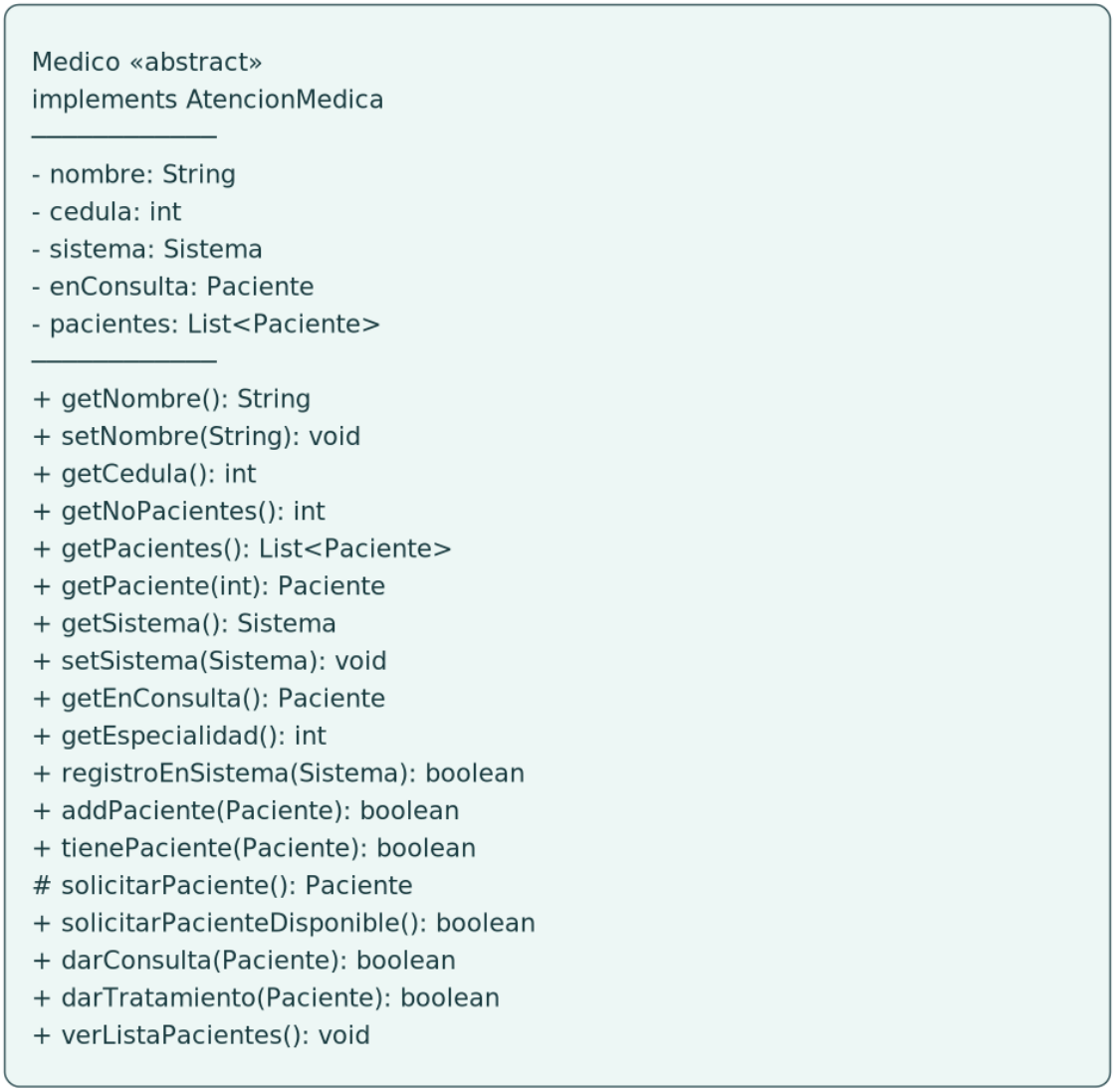


Figura 15: Diagrama de clase Medico.

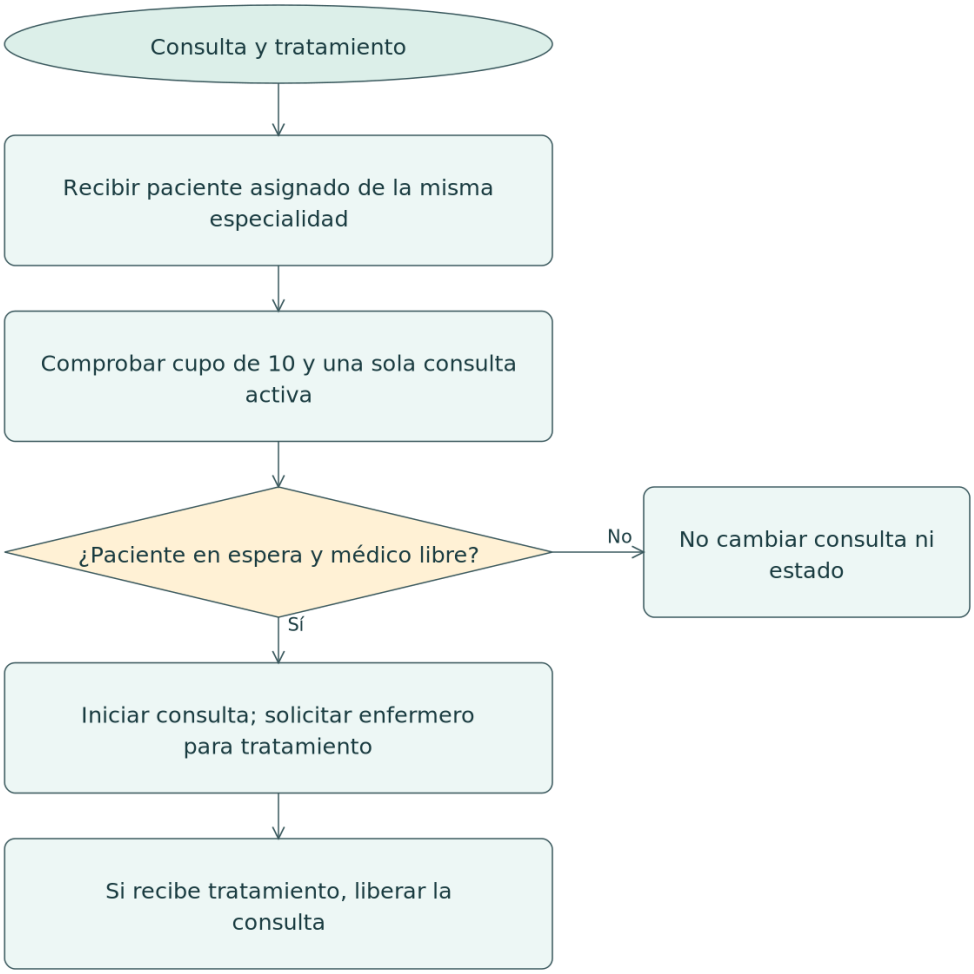


Figura 16: Flujo principal de Medico.

Pseudocódigo de Medico

```
ALGORITMO Medico
CONSULTA Y TRATAMIENTO

  Recibir paciente asignado de la misma especialidad
  Comprobar cupo de 10 y una sola consulta activa
  ¿Paciente en espera y médico libre?
  Iniciar consulta; solicitar enfermero para tratamiento
  Si recibe tratamiento, liberar la consulta
  Si no se cumplen las condiciones: No cambiar consulta ni estado
FIN
```

A.8. MedicoGeneral

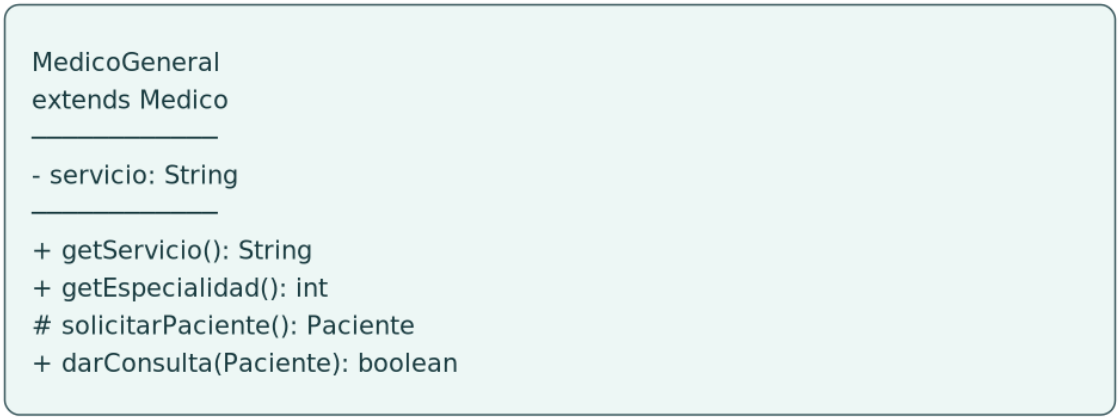


Figura 17: Diagrama de clase MedicoGeneral.

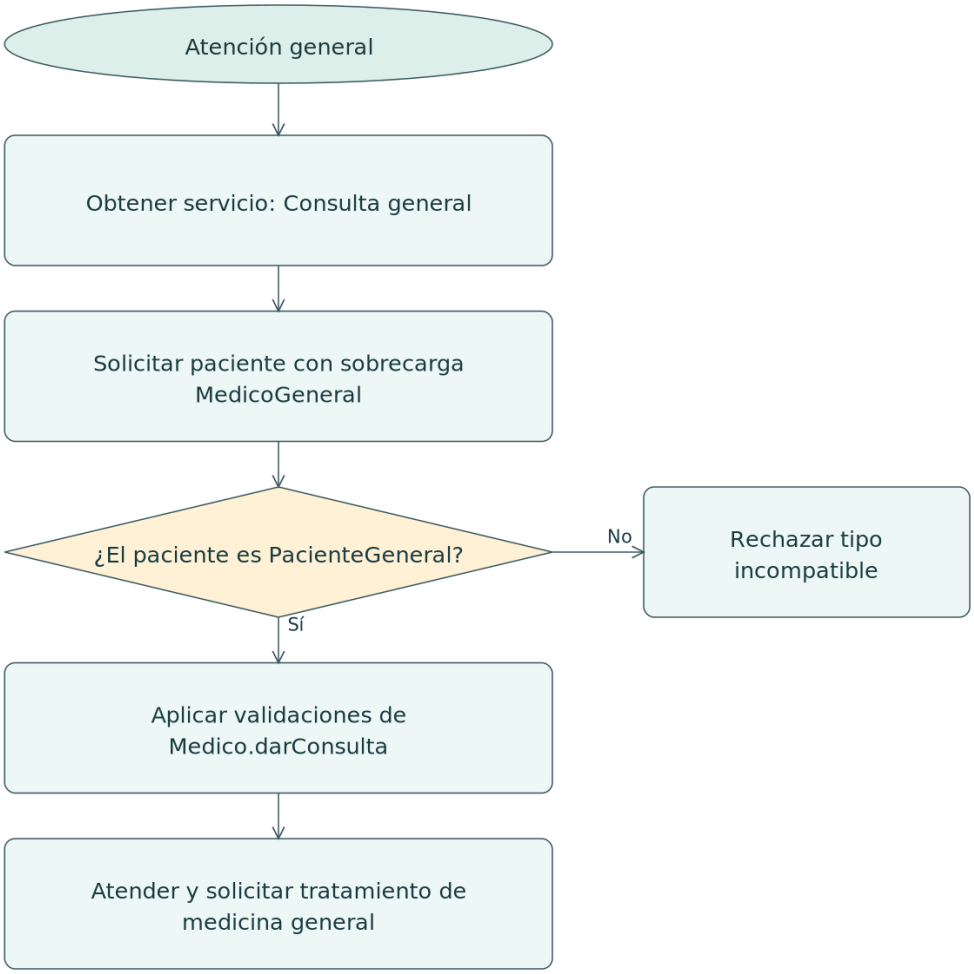


Figura 18: Flujo principal de MedicoGeneral.

Pseudocódigo de MedicoGeneral

```
ALGORITMO MedicoGeneral
ATENCIÓN GENERAL

  Obtener servicio: Consulta general
  Solicitar paciente con sobrecarga MedicoGeneral
  ¿El paciente es PacienteGeneral?
  Aplicar validaciones de Medico.darConsulta
  Atender y solicitar tratamiento de medicina general
  Si no se cumplen las condiciones: Rechazar tipo incompatible
FIN
```

A.9. MedicoEspecialista

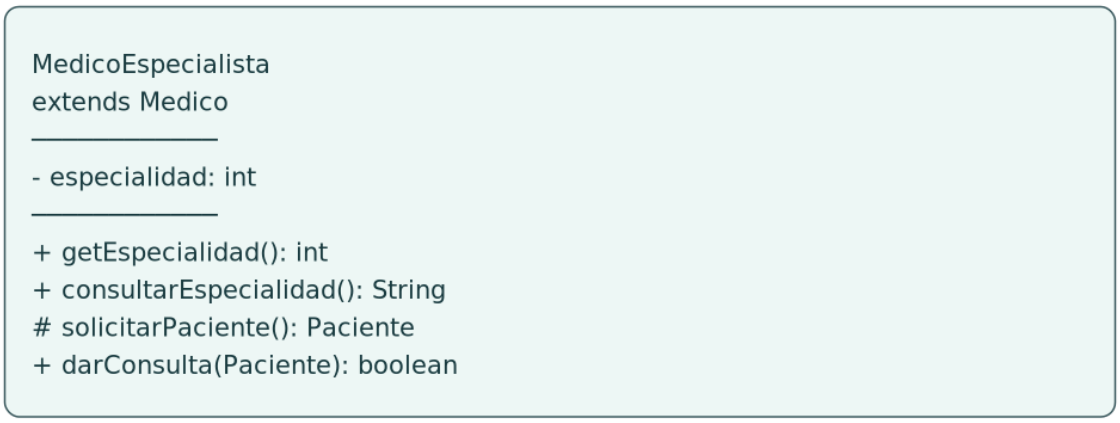


Figura 19: Diagrama de clase MedicoEspecialista.

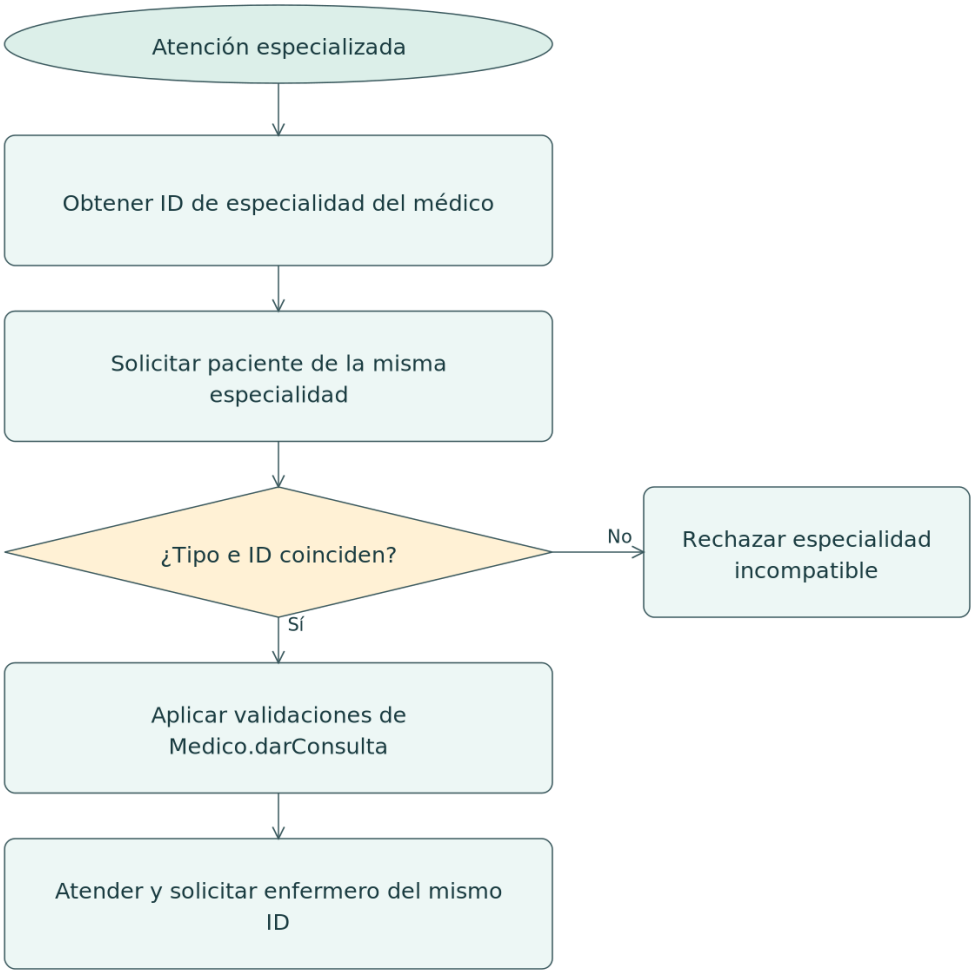


Figura 20: Flujo principal de MedicoEspecialista.

Pseudocódigo de MedicoEspecialista

```
ALGORITMO MedicoEspecialista
ATENCIÓN ESPECIALIZADA

  Obtener ID de especialidad del médico
  Solicitar paciente de la misma especialidad
  ¿Tipo e ID coinciden?
  Aplicar validaciones de Medico.darConsulta
  Atender y solicitar enfermero del mismo ID
  Si no se cumplen las condiciones: Rechazar especialidad incompatible
FIN
```

A.10. Enfermero



Figura 21: Diagrama de clase Enfermero.

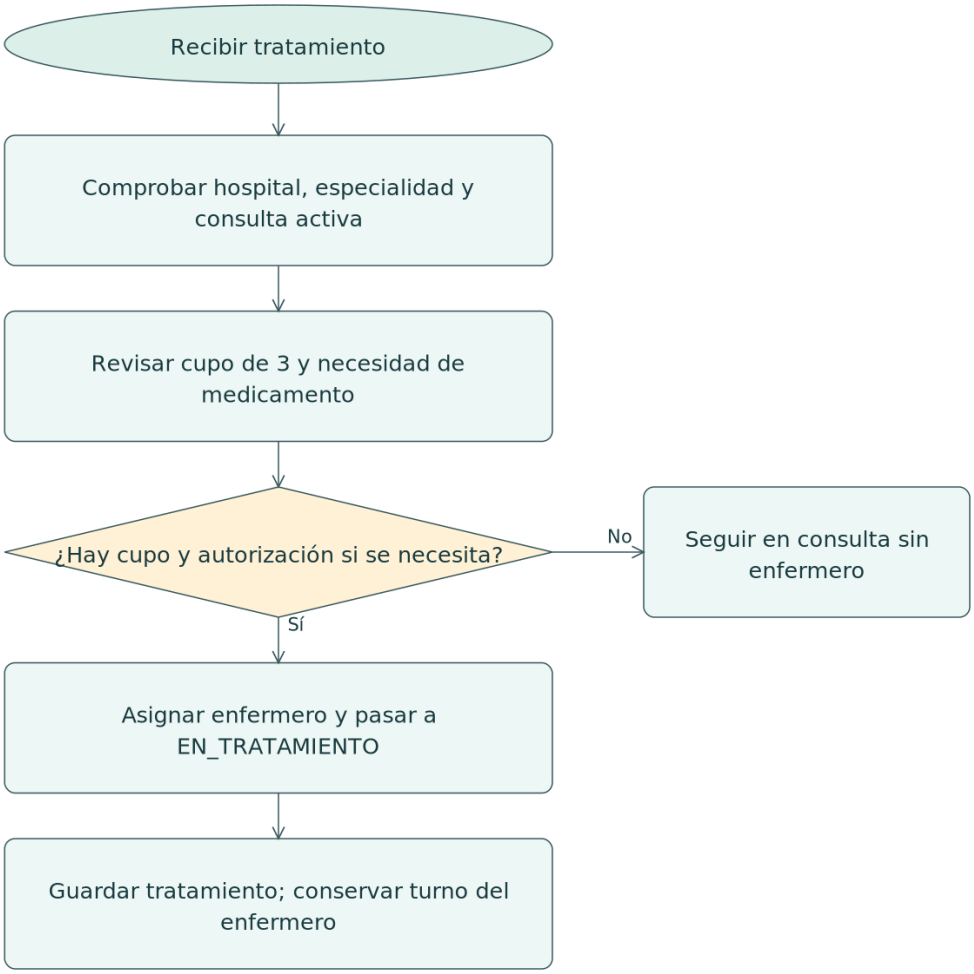


Figura 22: Flujo principal de Enfermero.

Pseudocódigo de Enfermero

```
ALGORITMO Enfermero
RECIBIR TRATAMIENTO

  Comprobar hospital, especialidad y consulta activa
  Revisar cupo de 3 y necesidad de medicamento
  ¿Hay cupo y autorización si se necesita?
  Asignar enfermero y pasar a EN_TRATAMIENTO
  Guardar tratamiento; conservar turno del enfermero
  Si no se cumplen las condiciones: Seguir en consulta sin enfermero
FIN
```

A.11. EnfermeroEspecialista

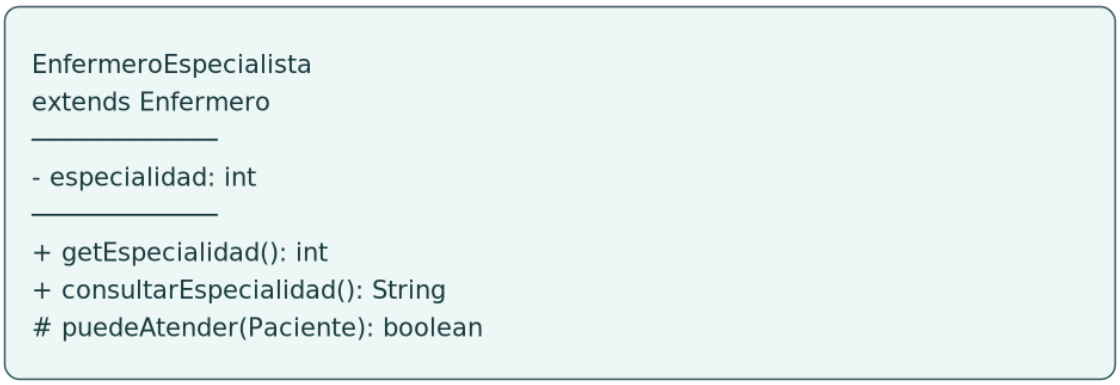


Figura 23: Diagrama de clase EnfermeroEspecialista.

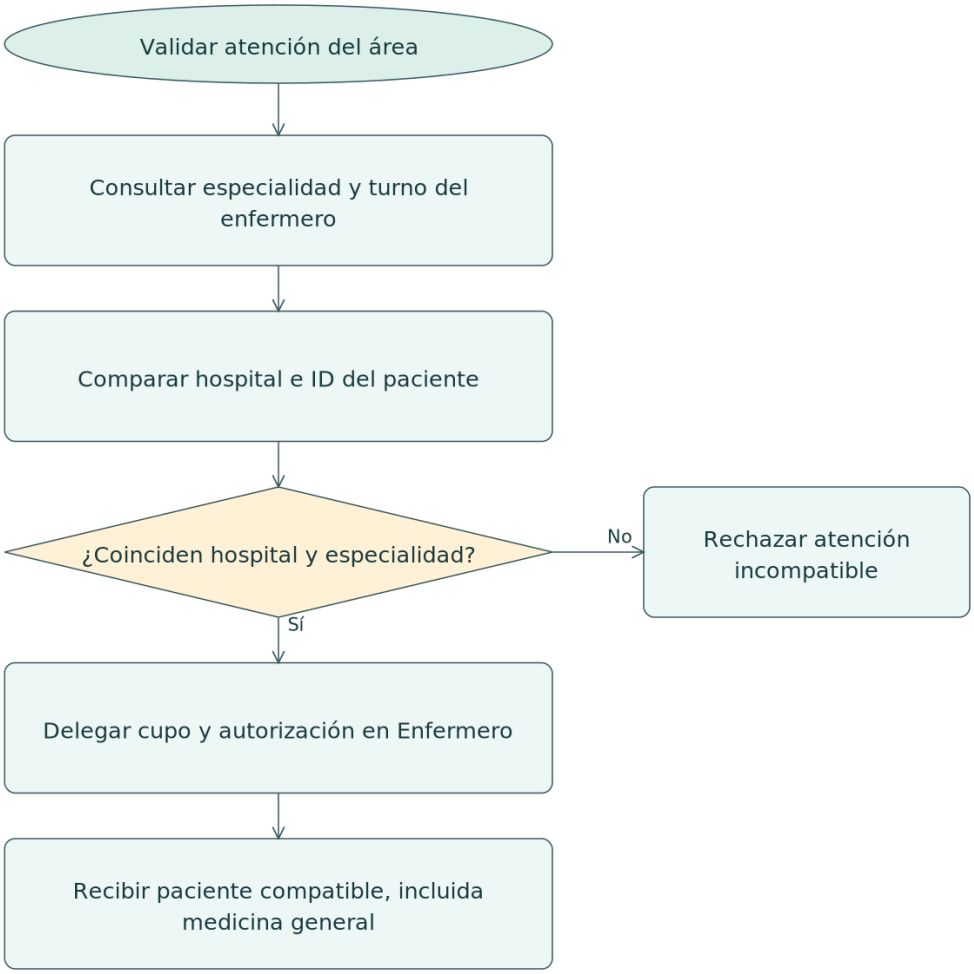


Figura 24: Flujo principal de EnfermeroEspecialista.

Pseudocódigo de EnfermeroEspecialista

```
ALGORITMO EnfermeroEspecialista
VALIDAR ATENCIÓN DEL ÁREA

  Consultar especialidad y turno del enfermero
  Comparar hospital e ID del paciente
  ¿Coinciden hospital y especialidad?
  Delegar cupo y autorización en Enfermero
  Recibir paciente compatible, incluida medicina general
  Si no se cumplen las condiciones: Rechazar atención incompatible
FIN
```

A.12. JefeEnfermeria

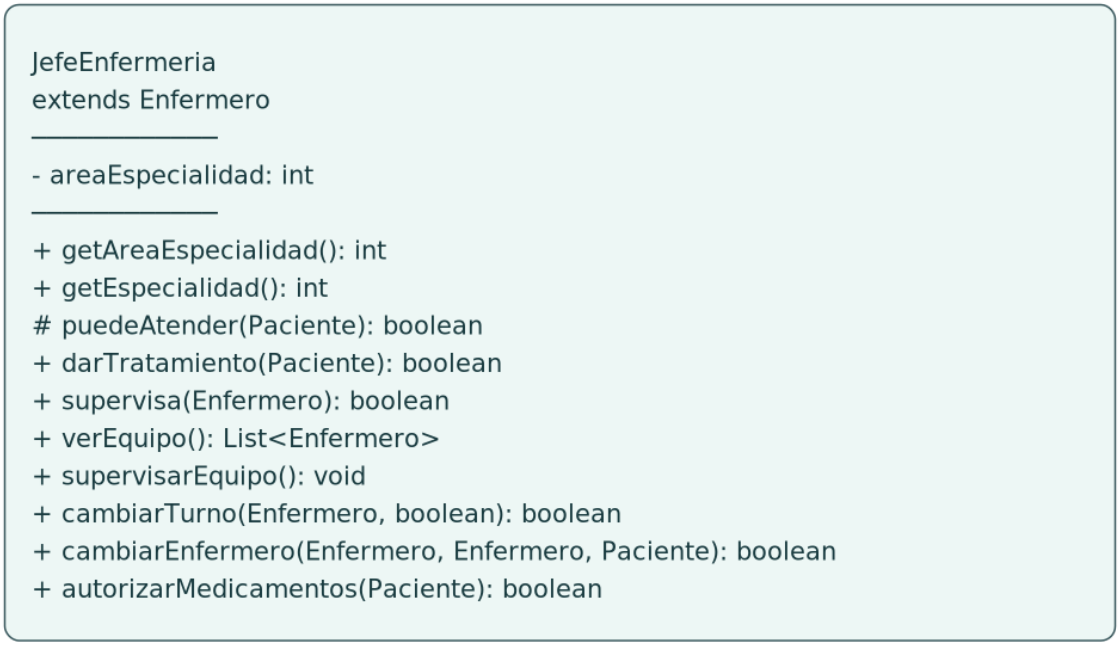


Figura 25: Diagrama de clase JefeEnfermeria.

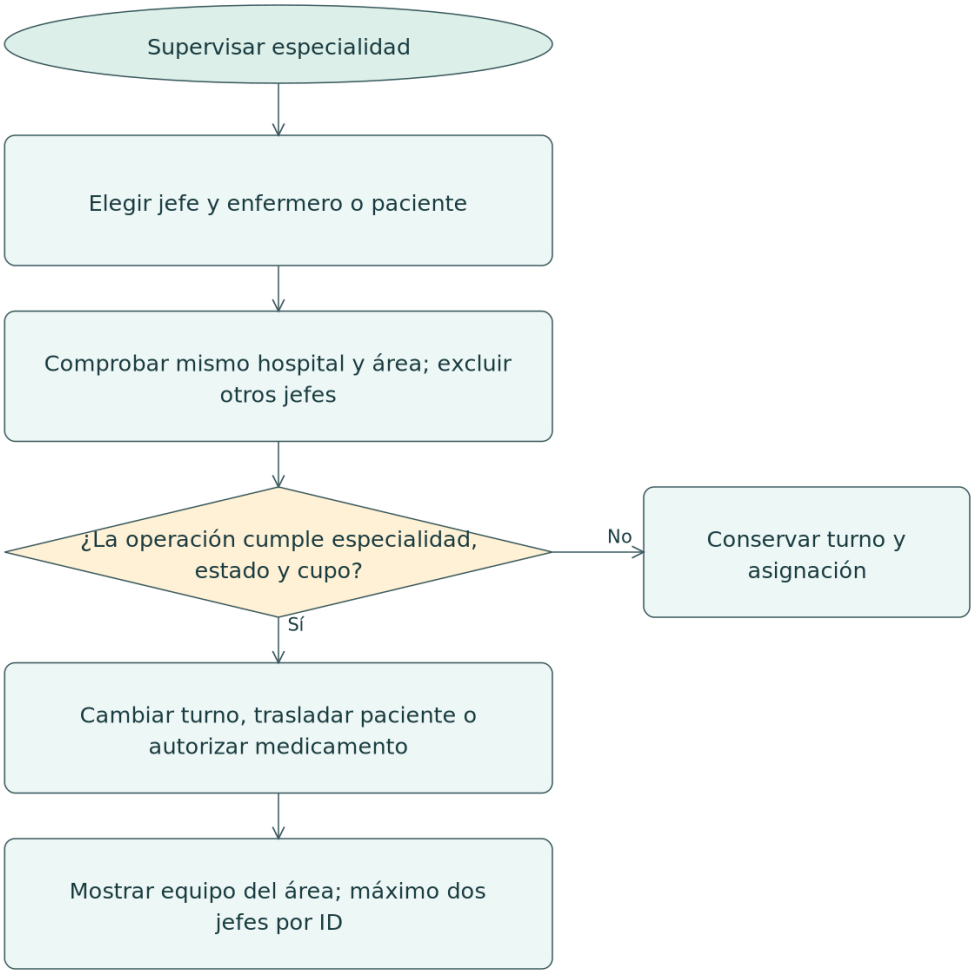


Figura 26: Flujo principal de JefeEnfermeria.

Pseudocódigo de JefeEnfermeria

```
ALGORITMO JefeEnfermeria
SUPERVISAR ESPECIALIDAD

  Elegir jefe y enfermero o paciente
  Comprobar mismo hospital y área; excluir otros jefes
  ¿La operación cumple especialidad, estado y cupo?
  Cambiar turno, trasladar paciente o autorizar medicamento
  Mostrar equipo del área; máximo dos jefes por ID
  Si no se cumplen las condiciones: Conservar turno y asignación
FIN

CAMBIAR ENFERMERO
  Validar supervisor, hospital, especialidad, paciente y cupo destino menor que 3.
  Si falla, conservar ambas listas y la asociación del paciente.
  Si cumple, retirar del actual, agregar al nuevo y actualizar el paciente.

AUTORIZAR MEDICAMENTO
  Si el paciente lo necesita y pertenece al área, autorizar siempre.
  Si falta un supervisor compatible, no iniciar el tratamiento.

CAMBIAR TURNO
  Verificar que el enfermero sea del área y no sea supervisor.
  Guardar false para matutino o true para vespertino.
```